

Yahoo finance data download - Parte 2

Continuiamo a vedere come utilizzare i dati Yahoo Finance attraverso la libreria 'yfinance':

nella prima parte [Yahoo Finance Data Download - Parte 1](#) abbiamo visto come lavorare su singoli ticker, in questa seconda parte vedremo come lavorare su più ticker effettuando download massivi.

A tale fine useremo delle liste di tickers raggruppati per tipologia contenute nei fogli di un file excel; le liste possono essere modificate a piacimento a seconda delle esigenze e degli obiettivi di analisi (alcuni ticker potrebbero risultare "scaduti" e non più disponibili, ma ciò non inficia il funzionamento del programma: semplicemente non ne viene effettuato il download)

I principali riferimenti sono quelli già riportati nella prima parte, quindi:

il sito del data provider, cioè Yahoo, dove potete andare a scegliere e i ticker che si intende scaricare:

- [yahoo finance](#)

quindi il link della libreria che andremo ad utilizzare:

1. [Yahoo Finance Python Library: yfinance](#) -

Il presente notebook può essere eseguito in Google Colab o in locale installando la libreria yfinance.

Nel prossimo episodio abbandoniamo l'open source e vedremo come utilizzare un notebook in Bquant di Bloomberg

*NB: In fase di download si possono riscontrare errori per i Ticker delisted senza che comunque l'esecuzione complessiva del codice venga bloccata. Per risolvere basta eliminare dai file di input tali Ticker. ***

```
In [ ]: !pip install yfinance
```

```
In [3]: # Import necessary libraries
import pandas as pd # Used for data manipulation and analysis
import numpy as np  # Used for numerical operations
```

```
import datetime      # Used for handling dates and times
import yfinance as yf # Used for downloading financial data from Yahoo Finance
#from yahoofinancials import YahooFinancials
```

```
In [38]: pd.set_option('display.float_format', '{:,.2f}'.format)
```

```
In [4]: def ImportData(FileName, sheets):
        """
        This function create a dictionary of lists of yahoo tickers
        Input variables:
        -FileName = .xlsx file containing the yahoo tickers grouped by type:
        a sheet for each type
        -sheets = it is a list variable containing the names of a the
        sheets in FileName
        Output:
        - DictData = it is a dictionary where each key corresponds to a list of tickers
        (the tickers in the sheet of FileName with the same name of the key)
        """
        GithubRepository = r'https://github.com/GMISSAGLIA/GM_PyLab/blob/Main/'
        DictData = dict()
        for Value in sheets:
            DictData[Value]= pd.read_excel(f'{GithubRepository}{FileName}?raw=true', f'{Value}')
        return DictData
```

```
In [5]: """ We import the Tickers to download from a .xlsx file available on github
        FileName is a string with the name of the .xlsx file on github
        The variable "sheets" is a list containing the names of the sheets in the .xlsx file (Variable "FileName")
        If you want a faster run or you are not interested in it, you can eliminate some items from the following list
        then we append all in a Dictionary """
        FileName = r'Yahoo_Selected_Tickers.xlsx'
        Sheets = ['WORLD_INDICES', 'CRYPTO', 'CURRENCIES', 'USA_TREASURY_RATES', 'TOP_STOCK', 'ETF']
        DictData = ImportData(FileName,Sheets)
```

```
In [ ]: """DicData is a dictionary of DataFrame containing Yahoo Tickers
        e.g. the key 'WORLD_INDICES' contains the DataFrame of World Stock indices
        DictData['WORLD_INDICES'] is a DataFrame"""
        type(DictData['WORLD_INDICES'])
```

```
In [7]: DictData['WORLD_INDICES']
```

Out[7]:

	Symbol	Name
0	^GSPC	S&P 500
1	^DJI	Dow 30
2	^IXIC	Nasdaq
3	^NYA	NYSE COMPOSITE (DJ)
4	^XAX	NYSE AMEX COMPOSITE INDEX
5	^RUT	Russell 2000
6	^VIX	CBOE Volatility Index
7	^FTSE	FTSE 100
8	^GDAXI	DAX PERFORMANCE-INDEX
9	^FCHI	CAC 40
10	^N100	Euronext 100 Index
11	^BFX	BEL 20
12	^N225	Nikkei 225
13	^HSI	HANG SENG INDEX
14	000001.SS	SSE Composite Index
15	399001.SZ	Shenzhen Index
16	^STI	STI Index
17	^AXJO	S&P/ASX 200
18	^AORD	ALL ORDINARIES
19	^BSESN	S&P BSE SENSEX
20	^JKSE	Jakarta Composite Index
21	^KLSE	FTSE Bursa Malaysia KLCI

	Symbol	Name
22	^KS11	KOSPI Composite Index
23	^TWII	TSEC weighted index
24	^GSPTSE	S&P/TSX Composite index
25	^BVSP	IBOVESPA
26	^MXX	IPC MEXICO
27	^MERV	MERVAL
28	^TA125.TA	TA-125

```
In [9]: """
We download from yahoo finance iterating through the dictionary DictData
and then through each DataFrame and we load the data in a Dictionary where
each key is a category and refers to a DataFrame containing
the values of each ticker : Open,      High,   Low,   Close,   Volume.
if you want try this code with a faster run you can choose a shorter period
"""

Dict_All_Quotes = {}
PERIOD = "max"
INTERVAL = "1d"
for key, value in DictData.items():
    """
    key identifies the type / group of ticker according to the sheet classification
    value is a DataFrame
    """
    tickers = list(value['Symbol'])

    Dict_All_Quotes[key] = yf.download(# or pdr.get_data_yahoo(...)
        # tickers list or string as well
        tickers = tickers,
        # use "period" instead of start/end - valid periods: 1d,5d,1mo,3mo,6mo,1y,2y,5y,10y,ytd,max
        # (optional, default is '1mo')
        period = PERIOD,
        # fetch data by interval (including intraday if period < 60 days)
```

```

# valid intervals: 1m,2m,5m,15m,30m,60m,90m,1h,1d,5d,1wk,1mo,3mo
# (optional, default is '1d')
interval = INTERVAL,

# group by ticker
# (optional, default is 'column')
group_by = 'ticker',

# adjust all OHLC automatically
# (optional, default is False)
auto_adjust = True

# download pre/post regular market hours data
# (optional, default is False)
#prepost = True,

# use threads for mass downloading? (True/False/Integer)
# (optional, default is True)
#threads = True,

# proxy URL scheme use use when downloading?
# (optional, default is None)
#proxy = None
)
Dict_All_Quotes[key].to_excel(r'Yahoo_' + str(key) + '_quotes_Matrix.xlsx')

```

```

[*****100%*****] 29 of 29 completed
[*****100%*****] 200 of 200 completed
[*****100%*****] 23 of 23 completed
[*****100%*****] 4 of 4 completed
[*****100%*****] 218 of 218 completed
ERROR:yfinance:
11 Failed downloads:
ERROR:yfinance:['IS', 'WE', 'FTCH', 'TWTR', 'FSR', 'AVLR', 'CPG', 'CS', 'WEBR', 'AUY', 'ATCO']: YFTzMissingError('$%ticker%: possibly delisted; No timezone found')
[*****100%*****] 480 of 480 completed

```

```

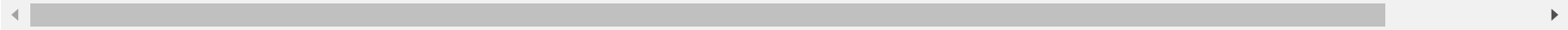
In [39]: #We have a two level column name: first the ticker name, second the data name : Open Price, High, Low, Close , Volume
Dict_All_Quotes['WORLD_INDICES'].head()

```

Out[39]:

Ticker	^NYA						^GSPC						^XAX							
	Price	Open	High	Low	Close	Volume	Open	High	Low	Close	Volume	...	Open	High	Low	Close	Volume	Open	High	Low
	Date																			
1927-12-30	NaN	NaN	NaN	NaN	NaN	NaN	17.66	17.66	17.66	17.66	0.00	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1928-01-03	NaN	NaN	NaN	NaN	NaN	NaN	17.76	17.76	17.76	17.76	0.00	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1928-01-04	NaN	NaN	NaN	NaN	NaN	NaN	17.72	17.72	17.72	17.72	0.00	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1928-01-05	NaN	NaN	NaN	NaN	NaN	NaN	17.55	17.55	17.55	17.55	0.00	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1928-01-06	NaN	NaN	NaN	NaN	NaN	NaN	17.66	17.66	17.66	17.66	0.00	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

5 rows × 145 columns



In [11]:

```
#we select our information using a tuple (tiker name, data name)
pd.DataFrame(Dict_All_Quotes['WORLD_INDICES'].head()[('^N100', 'Open')])
```

Out[11]:

^N100	
Open	
Date	
1927-12-30	NaN
1928-01-03	NaN
1928-01-04	NaN
1928-01-05	NaN
1928-01-06	NaN

```
In [ ]: #We select only World Stock Indices corresponding to the Key = 'INDICES'
#then we pick the key 'INDICES' to get our quotes
TickerType = "WORLD_INDICES"
df0 = Dict_All_Quotes[TickerType]
df0.describe()
```

```
In [ ]: #we pick oly the close price
DF_ALL = df0.loc[:, df0.columns.get_level_values(1).isin({"Close"})]
DF_ALL.columns = DF_ALL.columns.droplevel(1)
DF_ALL
```

```
In [ ]: DF_ALL = DF_ALL.fillna(method='ffill')
DF_ALL = DF_ALL.interpolate()
DF_ALL=DF_ALL[1:]
DF_ALL[DF_ALL != np.nan]
```

```
In [18]: df=df0.loc[:, ('^GSPC', 'Close')].copy(deep= True)
df.rename("PX_LAST", inplace=True)
df.index = df.index.rename('Date')
df
```

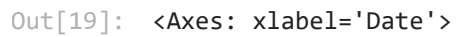
Out[18]:

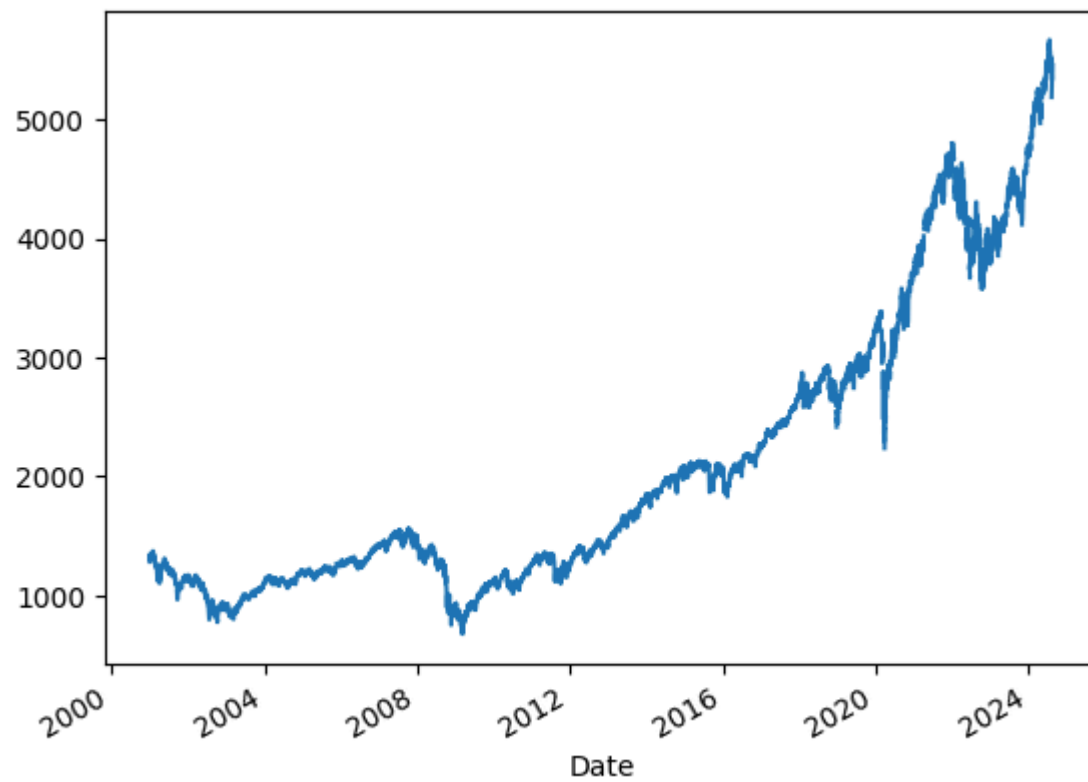
PX_LAST	
Date	
1927-12-30	17.660000
1928-01-03	17.760000
1928-01-04	17.719999
1928-01-05	17.549999
1928-01-06	17.660000
...	...
2024-08-11	NaN
2024-08-12	5344.390137
2024-08-13	5434.430176
2024-08-14	5455.209961
2024-08-15	NaN

25100 rows × 1 columns

dtype: float64

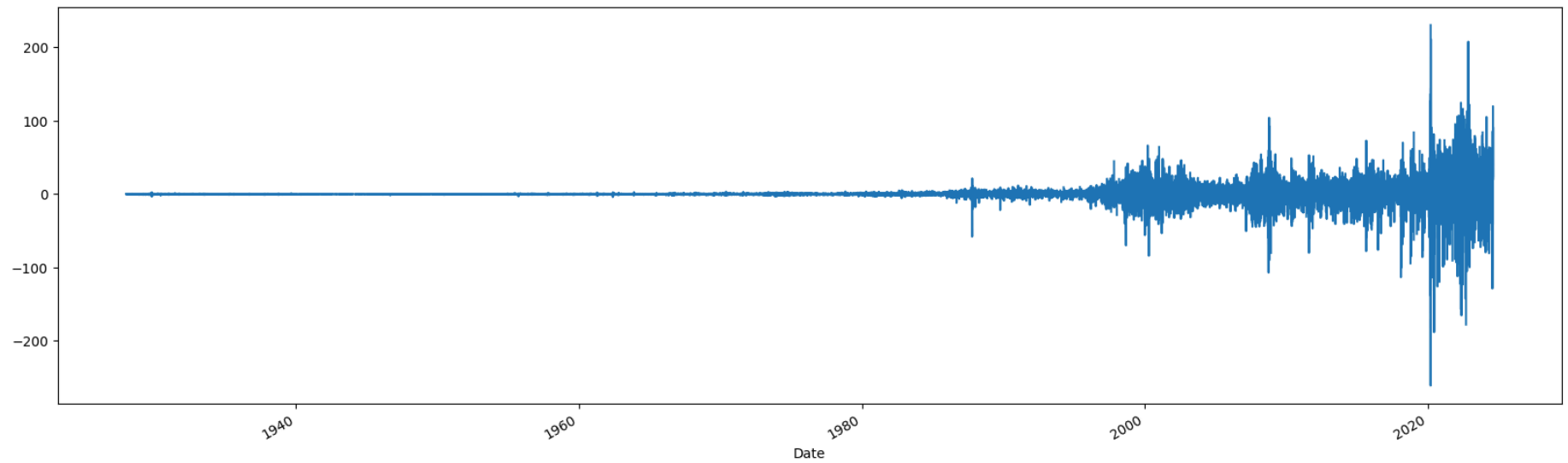
```
In [19]: import matplotlib.pyplot as plt
df.asfreq('M').plot() # asfreq method is used to convert a time series to a specified frequency. Here it is monthly frequency.
plt.title('SP500 MONTHLY')
plt.show()
df[df.index>datetime.datetime(2000,12,31)].plot()
```



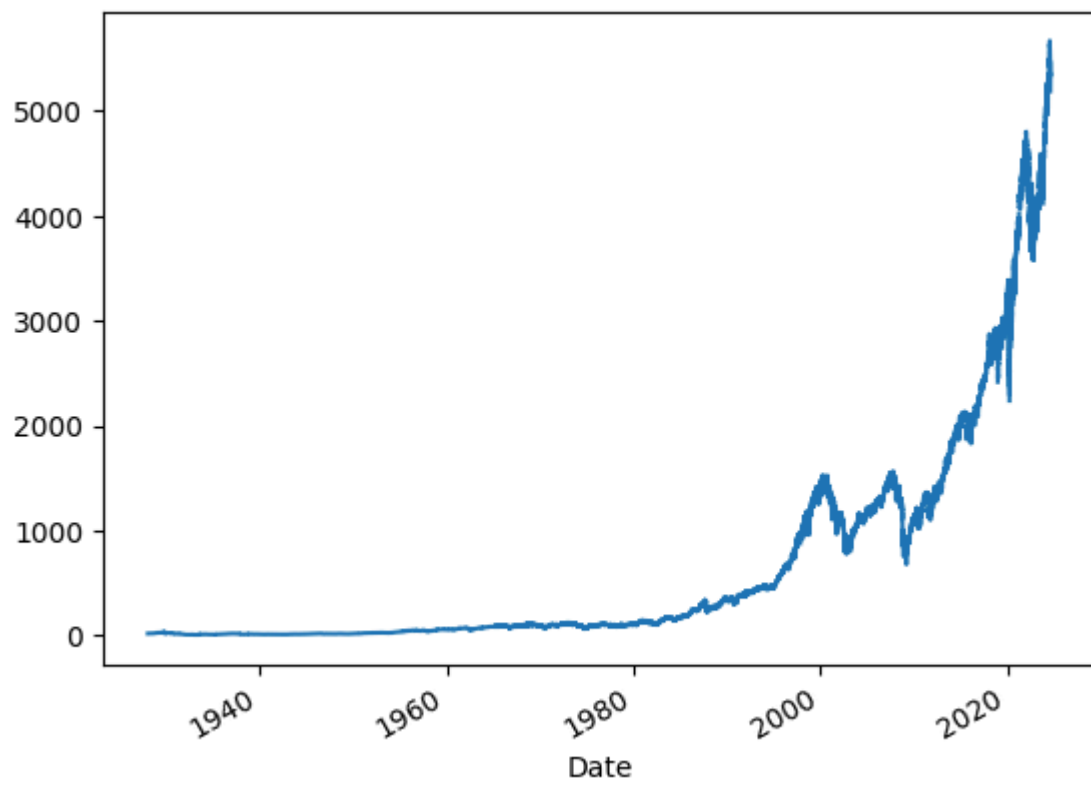


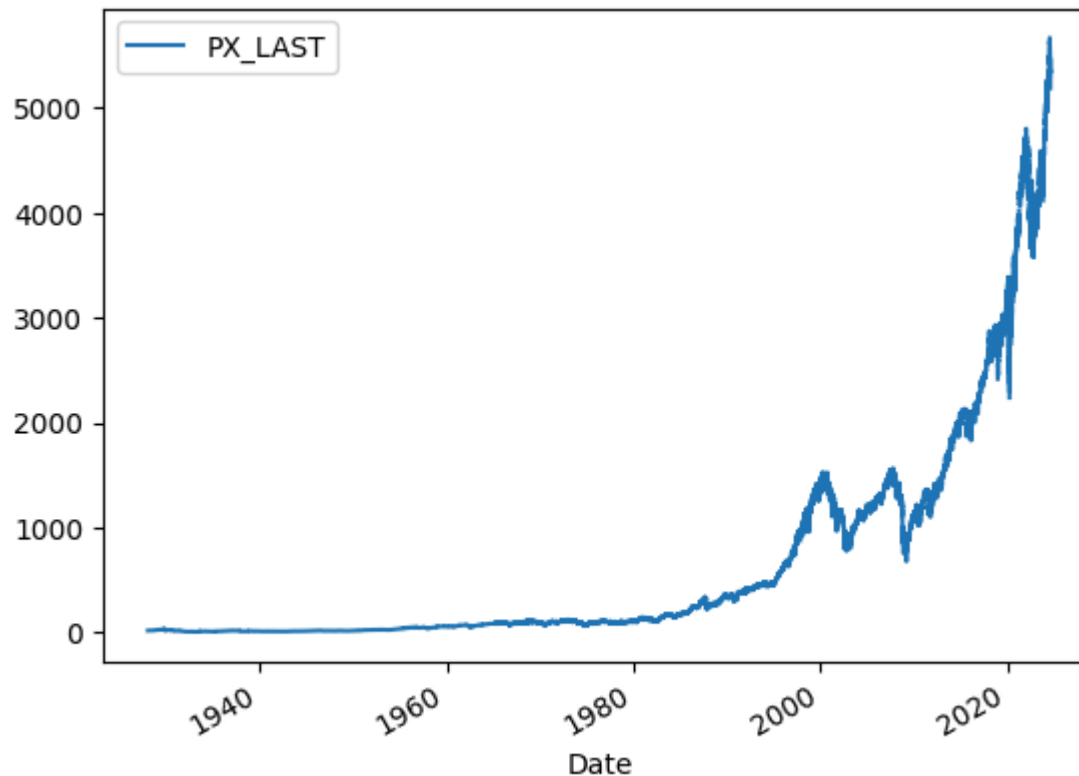
```
In [20]: DF = pd.DataFrame(df)
         DF.PX_LAST.diff().plot(figsize=(20,6))
```

```
Out[20]: <Axes: xlabel='Date'>
```



```
In [21]: rolling_DF = DF.PX_LAST.rolling('90D').mean()
DF.PX_LAST.plot()
DF.plot()
plt.legend(['PX_LAST', 'Rolling Mean'])
# Plotting a rolling mean of 90 day window with original High attribute of google stocks
plt.show()
```





Single Table Way

```
In [ ]: """here we download in a different way:
we append sequentially all the data marked by the corresponding ticker and category
in an unique DataFrame: DF_All_Quotes
with this approach there aren't null (NaN) values because each ticker has is full range of dates"""

PERIOD = 'max'
DF_All_Quotes = pd.DataFrame()
for key, value in DictData.items():
    tickers = list(value['Symbol'])
    for ticker in tickers:
        quotes = yf.download(ticker, period=PERIOD, interval = INTERVAL, auto_adjust = True, progress = False)
        quotes.insert(0, 'TICKER', ticker)
```

```
quotes.insert(0, 'Type', key)
DF_All_Quotes = pd.concat([DF_All_Quotes, quotes], ignore_index=True)
```

```
In [24]: DF_All_Quotes.drop(['Adj Close'], axis = 1, inplace = True)
```

```
In [25]: #.csv export
DF_All_Quotes.to_csv(R'DF_Yahoo_All_Quotes.csv')
```

```
In [26]: DF_All_Quotes.head(10)
```

```
Out[26]:
```

	Type	TICKER	Open	High	Low	Close	Volume
0	WORLD_INDICES	^GSPC	17.660000	17.660000	17.660000	17.660000	0.0
1	WORLD_INDICES	^GSPC	17.760000	17.760000	17.760000	17.760000	0.0
2	WORLD_INDICES	^GSPC	17.719999	17.719999	17.719999	17.719999	0.0
3	WORLD_INDICES	^GSPC	17.549999	17.549999	17.549999	17.549999	0.0
4	WORLD_INDICES	^GSPC	17.660000	17.660000	17.660000	17.660000	0.0
5	WORLD_INDICES	^GSPC	17.500000	17.500000	17.500000	17.500000	0.0
6	WORLD_INDICES	^GSPC	17.370001	17.370001	17.370001	17.370001	0.0
7	WORLD_INDICES	^GSPC	17.350000	17.350000	17.350000	17.350000	0.0
8	WORLD_INDICES	^GSPC	17.469999	17.469999	17.469999	17.469999	0.0
9	WORLD_INDICES	^GSPC	17.580000	17.580000	17.580000	17.580000	0.0

```
In [30]: DF_All_Quotes.describe()
```

Out[30]:

	Open	High	Low	Close	Volume
count	3,711,631.00	3,711,631.00	3,711,631.00	3,714,176.00	3,714,176.00
mean	740.89	749.19	733.10	740.94	153,099,976.60
std	10,490.48	10,694.54	10,320.90	10,514.30	59,200,861,469.32
min	-0.11	-0.10	-0.23	-0.10	0.00
25%	6.78	6.99	6.80	6.86	26,000.00
50%	21.73	22.00	21.63	21.79	654,800.00
75%	45.30	45.75	45.18	45.45	7,107,300.00
max	1,715,610.00	1,723,015.00	1,698,143.00	1,715,610.00	83,252,070,566,791.00

```
In [31]: DF_All_Quotes[DF_All_Quotes==np.nan].count()
```

Out[31]:

	0
Type	0
TICKER	0
Open	0
High	0
Low	0
Close	0
Volume	0

dtype: int64

```
In [32]: TickerType = "WORLD_INDICES"  
DF_All_Quotes[DF_All_Quotes.Type == TickerType].head()
```

```
Out[32]:
```

	Type	TICKER	Open	High	Low	Close	Volume
0	WORLD_INDICES	^GSPC	17.66	17.66	17.66	17.66	0.00
1	WORLD_INDICES	^GSPC	17.76	17.76	17.76	17.76	0.00
2	WORLD_INDICES	^GSPC	17.72	17.72	17.72	17.72	0.00
3	WORLD_INDICES	^GSPC	17.55	17.55	17.55	17.55	0.00
4	WORLD_INDICES	^GSPC	17.66	17.66	17.66	17.66	0.00

```
In [ ]: #Selected ticker
INDICES_TICKER_LIST = DF_All_Quotes[DF_All_Quotes.Type == TickerType]['TICKER'].unique()
pd.DataFrame(INDICES_TICKER_LIST)
```

```
In [34]: #Columns names
DF_All_Quotes[DF_All_Quotes.Type == TickerType].columns
```

```
Out[34]: Index(['Type', 'TICKER', 'Open', 'High', 'Low', 'Close', 'Volume'], dtype='object')
```

Intraday Data

```
In [ ]: All_Quotes_intraday = pd.DataFrame()
for key, value in DictData.items():
    tickers = list(value['Symbol'])
    for ticker in tickers:
        quotes = yf.download(ticker, period='5d', interval = '1m', auto_adjust = True, progress = False)
        quotes.insert(0, 'TICKER', ticker)
        quotes.insert(0, 'Type', key)
        All_Quotes_intraday = pd.concat([All_Quotes_intraday, quotes], ignore_index=True)
```

```
In [37]: All_Quotes_intraday.drop(columns = ['Adj Close'], axis = 1, inplace = True)
All_Quotes_intraday.to_csv(R'Yahoo_All_Quotes_intraday.csv')
All_Quotes_intraday.describe()
```

Out[37]:

	Open	High	Low	Close	Volume
count	1,522,333.00	1,522,333.00	1,522,333.00	1,522,333.00	1,522,333.00
mean	2,918.77	2,920.07	2,917.49	2,918.88	277,837.13
std	54,537.39	54,571.32	54,503.98	54,540.36	7,816,577.57
min	0.00	0.00	0.00	0.00	0.00
25%	0.90	0.90	0.90	0.90	0.00
50%	18.51	18.51	18.50	18.51	1,195.00
75%	65.41	65.42	65.39	65.41	9,872.00
max	1,627,572.38	1,629,039.62	1,625,933.50	1,627,911.25	3,165,004,071.00